University of Warwick

CS310 Computer Science Project 2018/19

JavaScript Based Video Editing Software

Author

Joshua Findon

Supervisor

Sara Kalvala

April 29, 2019

Abstract

HTML, CSS and JavaScript are fundamental to the creation of websites but have emerged as competent tools for building cross-platform software applications. By journeying through the process of developing video editing software for the browser, we dive into the world of web technologies to discover the wonders and hindrances of constructing a sophisticated web application. This undertaking has revealed the proficiency of JavaScript tools for building user interfaces, the various constraints of front-end applications and the power and versatility of WebAssembly.

Keywords: Video Editing, Software Development, Web Development, JavaScript, WebAssembly, FFmpeg

Table of Contents

1 Introduction **6**

2 Background **8**

2.1 Video Editing 8

2.1.1 iMovie 8

2.1.2 Premiere Pro 9

2.1.3 Clipchamp 9

2.1.4 Summary 11

3 Research **12**

3.1 Web Technologies 12

3.1.1 Web Workers 12

3.1.2 ArrayBuffer and TypedArray 12

3.1.3 SharedArrayBuffer 13

3.1.4 Canvas and WebGL 13

3.1.5 OffscreenCanvas 13

3.1.6 asm.js 14

3.1.7 WebAssembly 14

3.1.8 Emscripten 14

3.2 Gaussian Filter Experiment 15

3.2.1 Results 15

3.3 JavaScript Tools 16

3.3.1 Vue.js 16

3.3.2 Vuex 16

3.3.3 TypeScript 17

3.3.4 Webpack 17

3.4 Video Processing . . . 18
3.4.1 FFmpeg . . . 18
3.4.2 Alternatives . . . 18
3.5 Web Browsers . . . 19

4 Design **20**
4.1 Specification . . . 20
4.2 Application Strategy . . . 21
4.3 User Interface . . . 23
4.3.1 Storyboard . . . 23
4.3.2 Common Interface Components . . . 24
4.3.3 Layout . . . 24
4.3.4 User Guidance . . . 25
4.3.5 Aesthetic Properties . . . 26
4.4 Video Processing . . . 26
4.4.1 Format Support . . . 26

5 Implementation **28**
5.1 Data Management . . . 28
5.2 Video Processing . . . 29
5.2.1 FFmpeg Compilation . . . 29
5.2.2 Exporting with FFmpeg . . . 30
5.2.3 Previewing with HTML video and canvas . . . 31
5.3 User Interface . . . 33
5.3.1 Media Browser . . . 33
5.3.2 Timeline . . . 34
5.4 Documentation . . . 35
5.5 Networked Collaboration . . . 36

6 Testing **38**
6.1 Unit Testing . . . 38
6.2 Stress Testing . . . 38

7 Evaluation **40**
7.1 Project Management . . . 42

8 Further Work **43**
8.1 Feature Additions 43
8.1.1 Text and Transitions 43
8.1.2 JavaScript Plugins 43
8.1.3 Video Filters 43
8.2 Performance Improvements 44
8.3 Usability Improvements 45
8.3.1 Audio Preview Support 45
8.3.2 Data Persistence 45
8.3.3 Transformation Interface 45

9 Conclusions **46**

10 Acknowledgements **48**
10.1 Developer Resources 48

11 Glossary **49**

12 Legal Considerations **50**

Bibliography **51**

1 Introduction

There is an ongoing rise of software being developed primarily using HTML, CSS and JavaScript. These three web technologies were originally developed to create websites but through years of continuous evolution have become powerful tools for building user interfaces and general software development. These applications are cross-platform and benefit from a tremendous number of open source libraries and frameworks. User interfaces are expressed through the flexible layout capabilities of CSS. Web applications are often extendible by users through custom scripting. There are numerous advantages to building an application using HTML/CSS/JavaScript but this has not always been in true in the past.

In previous years, the appeal of adding advanced interactivity to websites lead to the existence of plugins such as Flash, Java and Silverlight. These plugins were responsible for video playback and better hardware utilisation but suffered from security issues and compatibility limitations between browsers. The advent of HTML5 eliminated the need for plugins for many cases by introducing video playback functionality and the canvas element for drawing graphics. Over time, the performance of JavaScript has improved with browsers implementing highly optimized interpreters and the number of web APIs available to browsers has grown. There are now powerful options available to make better use of hardware in a portable environment such as WebAssembly[1] and WebGL.

This motivated the idea to develop a substantially complex application based on web technologies and video editing software was a pertinent idea. Video editing is a computationally expensive process as it involves repeatedly executing a sequence of operations on large amounts of data, especially with high resolution media. In spite of the leaps of progress made by web technologies, there are still a multitude of limitations of writing an application with HTML/CSS/JavaScript. JavaScript as an interpreted programming language without access to low-level memory management cannot fully utilise hardware and is restricted by the APIs it has access to. This project will attempt to tackle the

challenges of developing resource-heavy software with the advantages and limitations of web-based applications by exploring current web technologies to enhance the software's features and performance.

2 Background

2.1 Video Editing

Video editing software is now a common category of software for manipulating and arranging video clips in the post-production phase of film-making. There are many different video editors available today including some which are developed using web technologies. Most web-based video editors rely on the server for video processing, this approach has its strengths and weaknesses but is severely limited by network speeds causing unresponsive video previewing.

The first two existing software that will be studied is Apple's iMovie[2] and Adobe Premiere Pro[3]. These are two popular video editors with opposing target audiences—iMovie is designed to be easy to use while Premiere Pro is suited for professionals. The third software to be explored is Clipchamp Create by Clipchamp[4], a video editor that runs in the browser. It was chosen because it features a client-side video encoder so it will be the most similar product to the one made for this project.

2.1.1 iMovie

iMovie is available for both macOS and iOS with its design heavily based around its ease of use which becomes apparent through its capabilities and limitations. It offers integration with other macOS and iOS applications so some of the user's media content is automatically presented and does not need to be imported. The software provides a set of preset titles, animated backgrounds and transitions for quick editing and offers some advanced features such as video stabilization and green screening but most of these are limited in customizability. iMovie attempts to offer helpful abstractions for some concepts, for example the length of clips in the timeline do not accurately portray their

length so that transitions can easily be identified and short clips are always distinguishable. Another example of this is the limit of two video layers and two audio layers so complex composition is not possible.

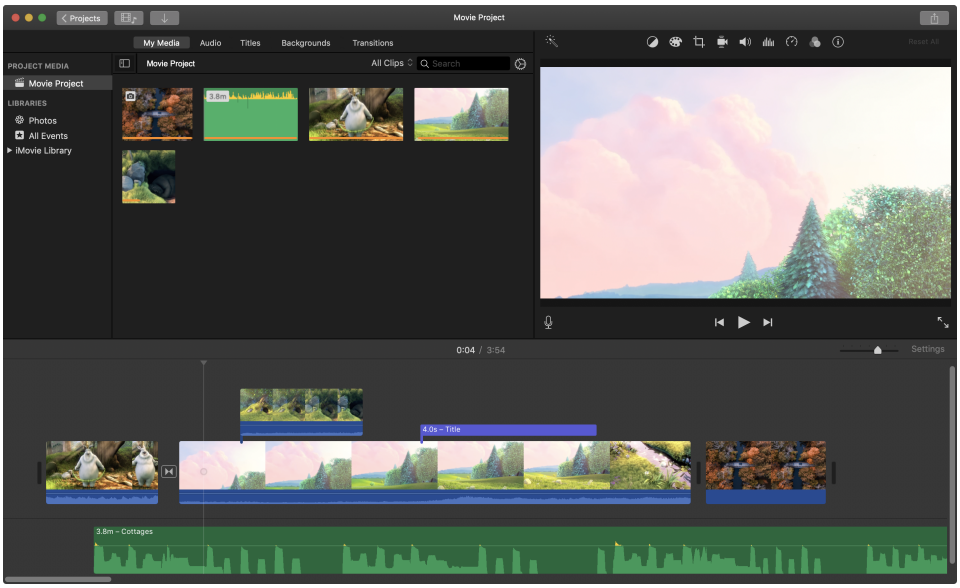

Figure 2.1: Screenshot of iMovie

2.1.2 Premiere Pro

Adobe Premiere Pro's suitability for professionals is evident through the extensive customizability throughout the program. The UI layout and keyboard shortcuts can be tailored by the user. Premiere supports many more file formats than iMovie with even more support available through plugins. There is a large variety of video effects and transitions that can be meticulously controlled using key frames. The overwhelming number of tools can make it difficult for new users to find simple editing functionality exemplifying the expense of full control for user experience. An example of this is the export menu which provides exhaustive details for the encoding format while iMovie only offers a selection of resolutions and quality presets.

2.1.3 Clipchamp

Clipchamp is a web application exclusive to Google Chrome with features similar to iMovie. These include text presets, transitions and video filters although in fewer quan-

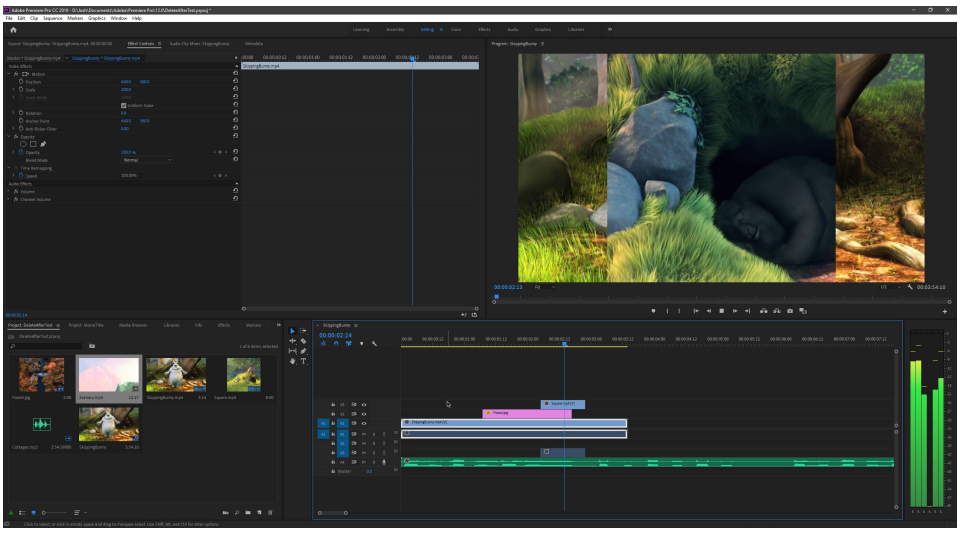

Figure 2.2: Screenshot of Adobe Premiere Pro

tity compared to iMovie. Unlike the other software, Clipchamp does not implement many desktop-like interfaces and is missing context menus, toolbars and includes very few keyboard shortcuts. Although Clipchamp does some of the processing locally, it still relies on the server for saving projects and other functionality so the software cannot be used offline. Clipchamp also has no plugin system for adding extra functionality, most likely because certain features such as HD video exporting are only available using the premium version.

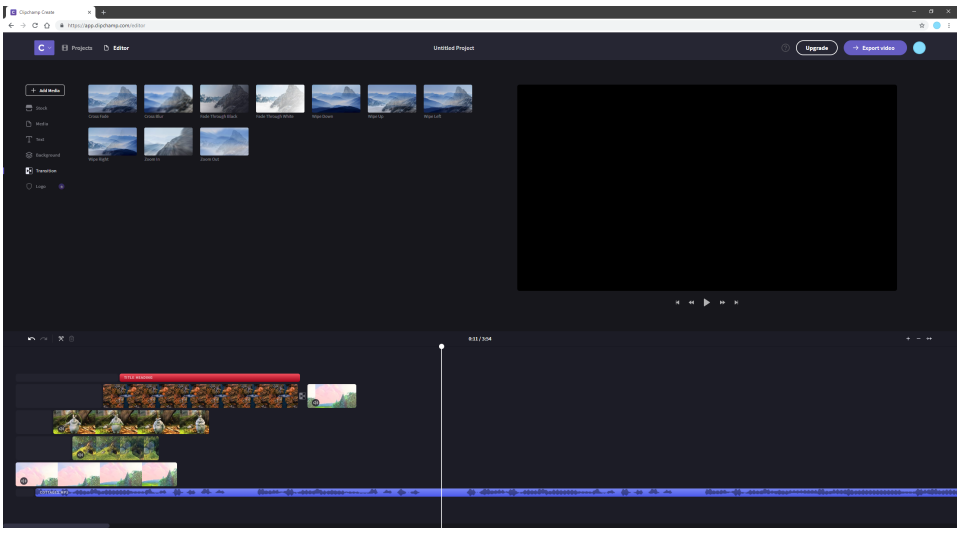

Figure 2.3: Screenshot of Clipchamp Create

2.1.4 Summary

Video editing software designed for different purposes focus on different details but there exist common concepts among all of the video editors. All three solutions make use of three main components in their user interface: an area for managing media, the timeline and an area for previewing the edited video. It is supposed that this a common interface layout because it clearly models the flow of information; the media, timeline and preview correspond to the input, processing and output of data throughout the editing process.

Each application also makes use of a timeline to graphically represent the video's content and a drag and drop system for adding elements to the timeline and reordering them. This interface attempts to provide an intuitive process for handling the complexity of video editing. The prevalence of these concepts indicate a proven foundation for video editing software so this project makes frequent use of these ideas.

3 Research

3.1 Web Technologies

This section explores and discusses performance enhancing web APIs and technologies, most of which were used in the project to some degree. The majority of the technologies below are considered as stable web features, the only exceptions are asm.js and OffscreenCanvas which is currently only supported in Chrome and asm.js. It is common for experimental features to rapidly change or become discontinued—sometimes these features are available through developer options in the browser. As none of the currently available experimental features were deemed significantly beneficial for video editing, the software was designed to function without any necessary modification of these options.

3.1.1 Web Workers

Web workers are a means of multi-threading JavaScript on the web, suited for background tasks. Worker threads run their own instance of JavaScript and communicate to other workers using message passing. Common concurrency problems are rarely encountered as workers are protected by limited access to thread safe APIs, notably there is no access to the DOM. The disadvantage of the message passing system is the communication restrictions it poses which can be inefficient in certain situations.

3.1.2 ArrayBuffer and TypedArray

An ArrayBuffer is a data structure capable of storing binary data, separate to the array feature that is apart of the JavaScript language. To access data inside an ArrayBuffer, a TypedArray interface is wrapped onto an ArrayBuffer, representing the binary data in

a specific representation. There are different TypedArrays for unsigned integers, signed integers and floats at varying byte sizes. ArrayBuffers allow data to be stored much more efficiently than other data structures in JavaScript and are therefore used by many APIs such as files, audio, video and WebSockets.

3.1.3 SharedArrayBuffer

A SharedArrayBuffer is a special type of ArrayBuffer than can be accessed by multiple web workers. This gives an option for much more efficient communication between workers compared to message passing, especially for large amounts of binary data. SharedArrayBuffer was disabled in many browsers because of security concerns when the Spectre vulnerability[5] was discovered. Chrome has since re-enabled support for this feature but it has setback the development of further threading technologies.

3.1.4 Canvas and WebGL

The HTML canvas element is a means for drawing hardware accelerated graphics to the screen through JavaScript. A canvas can be utilised through different types of rendering contexts which allow for varying levels of control. The 2D rendering context includes support for drawing two-dimensional shapes, text and images. The WebGL rendering context in comparison is more complex but powerful, based on the OpenGL ES 2.0 standard with support for 3D graphics and shaders[6].

3.1.5 OffscreenCanvas

This web API provides an interface to a canvas which is not displayed on the screen. Since this canvas is not associated with the DOM, it can be accessed directly by a web worker so drawing instructions can be executed on a separate thread. This can be used to perform expensive drawing calls that do not block the main thread—an important consideration for creating a responsive application because blocking the main thread disables scrolling and other interactions. The OffscreenCanvas can be used as a buffer by copying its contents to a regular canvas element when the drawing has been completed.

3.1.6 asm.js

asm.js is a low-level subset of JavaScript, designed to be a compilation target for software written in other low-level languages. There are two major advantages of this technology for web applications: the ability to run programs written in C/C++ and heightened performance. Support from the browser was not necessary for asm.js programs to run but execution is not guaranteed to be efficient unless the browser recognised the program as highly optimisable. Different browsers recognised asm.js differently and while a specification was in development for asm.js, it was not completed.

3.1.7 WebAssembly

WebAssembly (abbreviated Wasm) is an alternative approach to running more efficient code on the web. It is "a binary instruction format for a stack-based virtual machine"[7], that can run on multiple hardware architectures. It addresses many of asm.js's shortcomings by breaking free of its ties to the JavaScript language so WebAssembly modules are capable of better performance due to its more optimal representation. WebAssembly shares similarities with browser plugins but exists as an open web specification with a clearer focus on performance, portability and safety.

Game engines Unity[8] and Unreal Engine[9] support exporting to the browser through the use of WebAssembly and WebGL—this is a strong use case exemplifying WebAssembly's portability and performance. WebAssembly can also make use of threading by utilising web workers and SharedArrayBuffer, this has been demonstrated by a ray tracing example built in Rust[10]. While WebAssembly is suitable for porting programs made in languages with low-level memory models, support for languages with garbage collection is lacking. A garbage collection extension is being considered which would allow for easier integration with high level languages and access to the DOM[11].

3.1.8 Emscripten

Emscripten's description is a "toolchain for compiling to asm.js and WebAssembly, built using LLVM, that lets you run C and C++ on the web at near-native speed without plugins."[12] Emscripten[13] was originally devised to compile C/C++ to asm.js but the

project has since evolved to be able to output WebAssembly. Emscripten contains tools designed to take the place of existing C compilation tools so that a project may be compiled to a browser supported format with minimum refactoring.

3.2 Gaussian Filter Experiment

To test the performance and viability of implementing functionality with WebAssembly instead of JavaScript, a comparative experiment was carried out where the same algorithm was programmed in both languages. The specific task chosen was a Gaussian filter as it is a common image and video effect that can be implemented with little difficulty in JavaScript with the ImageData API. The hardness of the tasks can be trivially scaled up by inputting a larger image or kernel size.

The JavaScript version made use of array buffers to maximise memory performance. The ImageData API accepts an array buffer with colour information in RGBA order and the Gaussian blur was implemented as a convolution. The JavaScript was translated to C, many of the syntactical similarities for arithmetic, bitwise operations and arrays made this a straightforward task. The C program was then compiled to Wasm using Emscripten with the `EXPORTED_FUNCTIONS` compilation flag to make the function accessible from JavaScript. Array buffers were used to transfer the image data from JavaScript to WebAssembly and vice versa. The same browser, operating system and system specifications were used to carry out the experiment to eliminate external factors. Both variants are single-threaded for simplicity. A canvas element was used to display the input and output image data to ensure that the task was carried out successfully.

3.2.1 Results

The process was repeated 10 times on an image of size 1920x1080 and kernel size of 15x15. On average, the JavaScript version took 10.667 seconds to execute while the Wasm version took 3.798 seconds, a 281% increase in speed. It would be naive to assume that this statistic could be used to represent the general performance comparison of WebAssembly to JavaScript but this result does amplify the speed benefits of Wasm for tasks involving a large number of memory accesses and simple operations, supporting its practicality for use in video operations.

3.3 JavaScript Tools

3.3.1 Vue.js

It is common when developing a front-end application to use a framework which streamlines the implementation of common UI behaviours using an existing and proven architecture. Examples of these frameworks include Angular.js[14], React.js[15] and Vue.js[16]; these three examples are established around a component based architecture. This is where UI elements are written as individual, reusable components which each contain their own data and functionality. Although each of the three possibilities listed each have their own strengths and weaknesses, there was no identified characteristics which would benefit one over the others for this specific project. Vue.js was chosen because of the significant amount of prior experience with the library.

Vue components are self-contained modules composed from HTML, CSS and JavaScript to provide an organised architecture for both functionality and appearance. Vue.js enforces a particular communication system for interaction between components. A parent passes down data to children in the form of read-only "properties", any modifications to properties are reactively propagated down. A child passes up data to its parent by emitting events. The purpose of such a system is to simplify the data flow to a tree structure and de-couple the relationship between components to encourage reusability.

However, there are still many situations where communication needs to be interconnected and propagating all data through the tree would be superfluous. A common solution to this problem is the use of a global state. Data that needs to be used in multiple places is updated through events to the global state. This can be managed manually for a simple application but data flow can become chaotic with many components. The companion library Vuex[17] provides a state management pattern to solve this problem.

3.3.2 Vuex

Vuex is influenced by the flux state management architecture[18] which imposes strict rules for storing data focusing on a unidirectional data flow. Vuex relies on two important principles: a single source of truth is used to keep the state in one central location and

modifications to the state must be made through predetermined "mutation" functions. Mutations must be synchronous to keep the order of modifications deterministic. Asynchronous behaviour can be implemented through "actions", which can make synchronous calls to mutations.

Any component can call any mutation or action and listen to any updates to the state. Vuex's verbose representation of state mutations encourage clear state modifications to ensure the state is consistent and traceable. Vue.js's debugging tools support observation and editing of Vuex stores and allow for time-travel debugging.

3.3.3 TypeScript

One option which was thoroughly considered was to use the TypeScript[19] language, a typed superset of JavaScript that compiles to plain JavaScript, developed by Microsoft. TypeScript's main purpose is to bring (optionally enforced) static typing to JavaScript. TypeScript boasts a powerful typing system supporting generics and variant types and also offers many additional language features such as enums and interfaces.

Using TypeScript benefits the scalability of the software as it adheres to object oriented programming principles much more closely. This especially benefits teams as it promotes type-based interfaces and aids code comprehension. Enforcing types also guarantees type safety in many situations which leads to more reliable code and an easier debugging experience.

Despite these advantages, TypeScript was not selected for this project due to concerns with inflating development costs and lack of experience. This was a decision that was required to be made early as using TypeScript would affect the architecture of many systems.

3.3.4 Webpack

Vue CLI is a system for setting up Vue projects with auxiliary development tools, the primary one being Webpack[20]. Webpack bundles JavaScript and utilised dependencies into a single script and organises static assets for deployment—it can execute this process automatically by watching for file changes. It integrates with Vue to provide a hot

module replacement feature so that updates can be made without restarting the software and components will retain their state for uninhibited debugging.

3.4 Video Processing

3.4.1 FFmpeg

FFmpeg is an open source library built for operating on video and audio on a vast variety of file formats. It was desirable to use this library to implement complicated video operations given the restricted development time so that the end result would include a larger range of features. FFmpeg was the perfect candidate as it includes an extensive number of features and importantly has the functionality needed for implementing features from the specification—merging and concatenation of audio and video, video transformations and text overlays. FFmpeg is relied upon by numerous software and its popularity is a testament to its dependability and has lead to an abundance of online resources.

FFmpeg is written in C so it cannot be run straightforwardly in the browser, there are however successful attempts of compiling FFmpeg for the web. The ffmpeg.js[21] project compiles FFmpeg, including many of its dependencies, to asm.js using Emscripten. The original plan was to use this pre-compiled solution to implement the video operations for the software. It is discussed during the implementation section as to the reasons behind avoiding this approach and compiling FFmpeg using Emscripten from scratch.

3.4.2 Alternatives

Other approaches were also researched to confirm that using FFmpeg was the most fitting option. Another motivation was the possibility to switch to one of these alternative options if a difficulty was encountered. These alternatives were not used during development but could potentially be used for future development of features.

CCapture.js[22] is a JavaScript library for recording video drawn to the canvas element by intercepting timing calls. This could be used as an alternative to FFmpeg for exporting videos but has very limited format support. Broadway.js[23] is a JavaScript H.264 decoder and player that can be used to decode videos without using the DOM,

made using Emscripten and WebGL. This allows decoding to be allocated to an individual thread but is unlikely to be as fast as the HTML video element as it is written in JavaScript.

3.5 Web Browsers

Different browsers have varying levels of support for web APIs. For instance, the video tracks API for accessing different video layers within the same video file is supported in Microsoft Edge and Apple's Safari but not in Mozilla FireFox or Google Chrome. After evaluating the degrees of support in these browsers, Chrome was considered as having the most encompassing coverage for features which were valuable to this project such as SharedArrayBuffers and OffscreenCanvas.

4 Design

4.1 Specification

The application was named "V Editor". A brief version of the specification for V Editor is outlined below. Nested items were included after the completion of the original specification to provide more detailed coverage of the features implemented.

1. Graphic User Interface

- Toolbar
- Context Menu
- Drag and Drop
- Keyboard Shortcuts
- Dropdown

2. Creation, saving and loading of video projects

- Ability to link media to a source file if it is missing

3. Import media supporting H.264 video, JPG/PNG images and MP3/AAC/PCM audio formats

- Ability to preview media and their properties

4. Timeline controls for moving and trimming video and audio across layers

- Scaling of the timeline
- Timeline ruler

- Copy/Paste clips
- Undo/Redo changes

5. Up-to-date previewing of the video
6. Export H.264 video up to 60 FPS and 1920x1080 resolution
7. Text can be added as a video clip with customizable font and size
8. Translation and scaling transformations for video clips
9. Visual transitions between clips

Objectives 7 and 9 were not completed so are absent from the Implementation section. Objective 8 was implemented before 7 because it was deemed more plausible given the remaining time at that point in development.

4.2 Application Strategy

There are a variety of options available for serving a HTML/CSS/JavaScript application. A client-server model presents the interface in the browser while the server manages data storage and expensive computations. For this approach, there would be certain difficulties to consider such as the case that the server was in a remote location, a video editor would encounter difficulties with networked data transfer speeds and could harm the application's responsiveness. Another common approach is to embed the HTML/CSS/JavaScript inside a native desktop application.

The Chromium Embedded Framework[24] can be integrated into desktop applications to utilise Chrome's web browsing capabilities, allowing any part of the interface to be controlled using HTML/CSS/JavaScript. Electron is a more recent framework which bundles Chrome and Node.js, allowing web applications to be packaged with minimal configuration. Electron offers JavaScript APIs for desktop application functionality like windowing and notifications. Node.js can be used to access the file system, processes, network and includes the ability to interface with C/C++ programs. The main allure of this type of a desktop-based application is the possibility of full hardware utilisation while maintaining many of the advantages of web applications. On the contrary, the drawback is that desktop applications must be trusted and installed onto a users system and the inclusion of an entire browser inflates the program's file size.

The approach chosen for this project was to develop an entirely front-end application to run in the browser with minimal reliance on the server, the most restrictive of the approaches presented. The client-server and desktop approaches already demonstrate solutions for persistent data storage and the prospect for executing computationally expensive tasks in C or any other method available on the system. However, these approaches invalidate the usefulness of technologies like WebAssembly so to cultivate a more comprehensive look at web technologies, the front-end approach was taken. This decision also acted as insurance for the case that a significant problem instigated an alternative approach. Adding server support or switching to Electron would be more straightforward than vice versa as some of the implementation would need to be revised.

Another option would be to combine multiple approaches by compiling multiple versions of the project. Tools like Figma[25] offer both web and desktop applications to improve the onboarding experience while still giving the option for desktop app features. This approach was disregarded for this project as these advantages come at the cost of development time with features like data storage needing multiple implementations of the same system.

The desktop version of Chrome was selected as the target platform. Supporting a single browser omits the difficulties of cross-browser incompatibilities and would diminish the number of web APIs available for use. Mobile support was not considered due to feature incompatibilities, extra UI obstacles and performance concerns. Other benefits included the availability of Chrome on Windows, macOS and Linux ensuring the software was not constrained to particular operating system and the guarantee that the software could be packaged with Electron in the future.

The notion of a Progressive Web Application (PWA)[26] has similar to goals to this project—a web application which provides more native-like features and speed. A PWA is characterized by its support for modern APIs such as “service workers”, this API gives the developer control over the caching of network resources to allow the application to function offline. While the idea of a PWA has more benefits as an alternative to mobile application, Chrome also later added support for desktop PWAs[27]. PWA philosophies were not employed for this project as video editor functionality was considered a larger focus in comparison to the optimization of online loading times.

4.3 User Interface

The application's primary audience is someone with casual video editing experience but with consideration for new users. In comparison to the video editors looked at in the background section, the editor will try to offer a middle ground experience between iMovie and Premiere Pro. The user interface is therefore designed to strike a balance between being unobtrusive to experience users while staying friendly for beginners. Many design elements were influenced from other video editors and software but only with clear justification.

4.3.1 Storyboard

A storyboarding technique was used to visualise the flow of the interface, illustrated in Figure 4.1. There are two main screens in the interface, the welcome page and the main timeline view. The remaining nodes are dialog boxes for importing and exporting data. All dialog boxes are modal, as in the remainder of the UI cannot be accessed until the user has finished interaction.

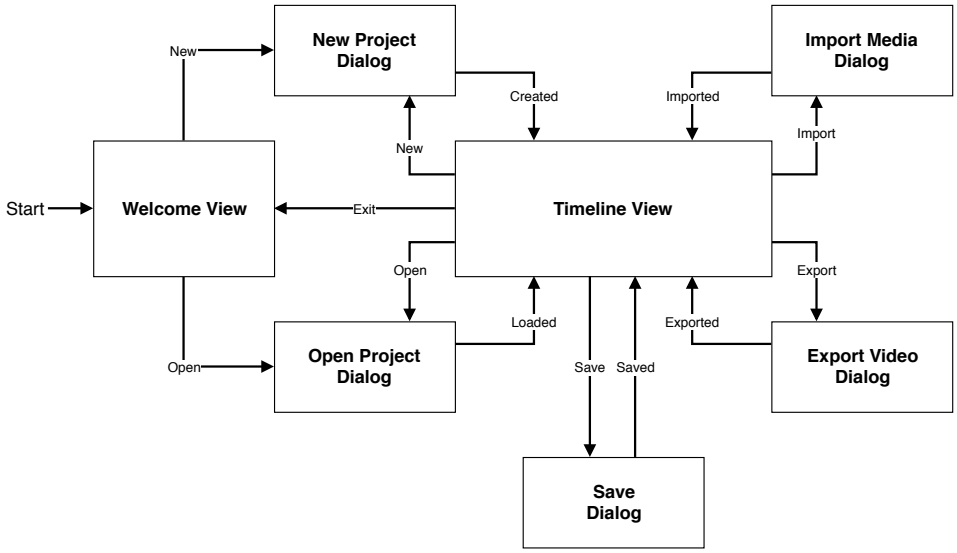

Figure 4.1: User Interface Storyboard

4.3.2 Common Interface Components

The design of the interface borrows many elements from other desktop applications: drag and drop, toolbars,context menus and keyboard shortcuts. These are accustomed concepts to users familiar with other programs and give the users a logical starting point for understanding the interface.

The toolbar, a collection of categorized dropdown operations prominent in video editors, was a sensible space to house many of the application's operations. Alternatives to a toolbar were contemplated but were considered inferior. Individual text buttons for each operation would be overly spacious while icon buttons would require the user to memorize and learn new symbols. Keyboard shortcuts work well in tandem with a toolbar as it allows users to discover them through regular use but are inhospitable for new users when it is the sole option.

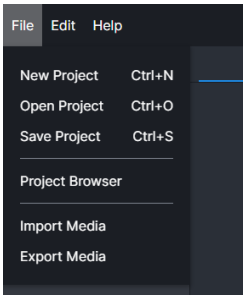

Figure 4.2: Screenshot of the implemented toolbar

4.3.3 Layout

The design of the user interface was fundamentally based around the the three main components of a video editing UI analysed in the Background, Summary section—the media browser, timeline and video preview. The positions of these elements are also logically placed in a similar position to other video editors.

The video preview will need a similar aspect ratio as the entire screen so it takes up 1/4 of the interface and is positioned in a corner. The timeline ideally uses as much space as possible to display the video contents across time. A larger timeline allows for a better overview of the video project and more precise adjustments. Since desktop

screens are on average larger horizontally than vertically, the timeline is allocated the full width of the screen. The media browser displays an adaptively sized grid of items and while a larger size would be capable of displaying more content, it is not as critical as the timeline. Along with this reasoning, the positions of these components follow the conventions of other video editors to keep the interface more recognisable.

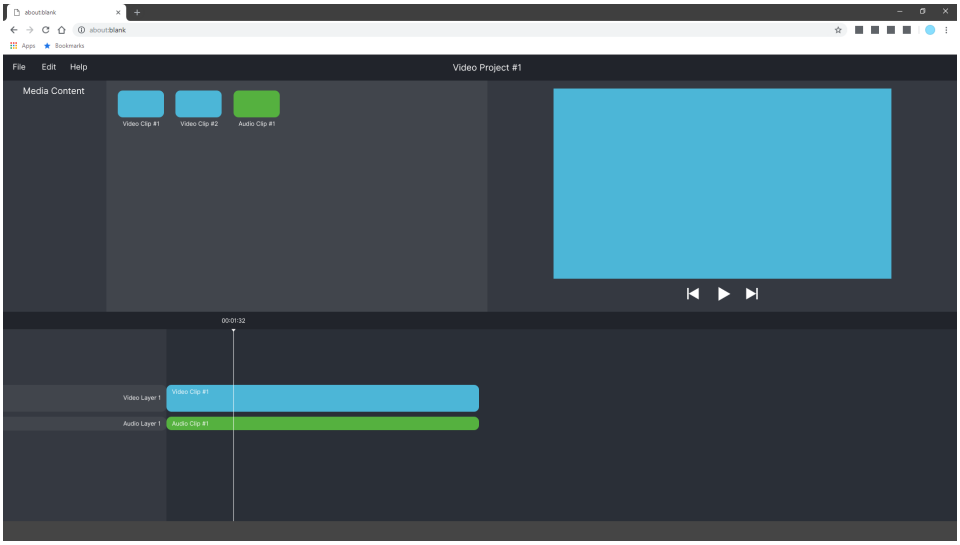

Figure 4.3: A mock-up prototype of the timeline view.

To add functionality for modifying the position and resizing of video, a subsequent change was made to the layout by utilizing tabs for the media browser and the clip details panels. The clip details panel includes input boxes for editing position and size values. Rather than replacing other parts of the interface, tabs were used to hide the media browser so clips could still be selected in the timeline and the preview can be used to observe changes to the values. One of the sacrifices made by this design choice was that the media browser is no longer always available and the user must recognise that these tabs must be used to access the separated functionality.

4.3.4 User Guidance

Two systems were designed to educate users on how to use the software. The welcome page includes a button to access tutorial mode—this mode walks through the various UI components presenting an accompanying explanation while giving a general guide for how the user can begin to import media and start editing. A help centre, available

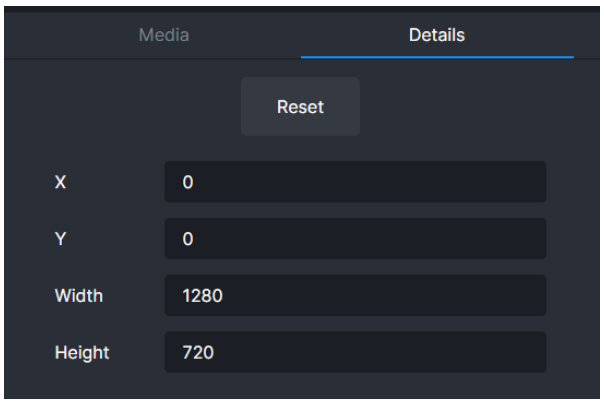

Figure 4.4: The top left of the interface displaying the clip details panel.

in the toolbar, contains guidance on more specific topics for achieving common tasks.

4.3.5 Aesthetic Properties

The interface primarily uses the Inter typeface, a free, open source font designed for computer screens. The interface also features a dark colour scheme, a common choice for video related software as the video content stands out more compared to the interface. Interface components and custom icons were designed manually to achieve a consistent style.

4.4 Video Processing

Video decoding is vital for two features of the software, video previewing and video exporting. Both systems require decoding input files but are distinguished by their output—previewing involves rendering to the screen while exporting requires encoding a video file. The design and justification of these systems is deliberated in the implementation section.

4.4.1 Format Support

Video and audio formats are split into two categories, the compression format and the container format. The compression format describes the representation of the data and

dictates which algorithms are required to encode and decode this data. The container format describes the structure of the file and is responsible for organising the content and metadata; a file's container format is indicated by its file extension. For example, a video encoded using the H.264 compression format may be stored in an MP4, MOV or MKV container format. Video files with the same compression format may also have different bit rates, frame rates, resolutions and colour encoding.

A number of different video compression formats were considered but the software was designed to only support the H.264 format with the MP4 container format. This decision was supported by three main factors; it is currently a common video format supported by many existing video editors and players, it is supported by ffmpeg.js and it has native playback support in browsers. It also strikes a good balance between decoding/encoding time and file size. It may have also been reasonable to support the VP8/9 compression format and accompanying WebM container format which is also supported by ffmpeg.js and the browser, however it was dismissed after concerns that the benefits and lessons learned would not outweigh the increased time spent testing. For maintainability reasons, minor provisions were still made to ensure additional format support is possible in the future.

Upcoming compression formats which were also considered were H.265 and AV1. H.265 is intended as a successor to H.264, offering better compression sizes but is not supported in Chrome due to issues with efficiency and licensing[28]. It is common for video compression techniques to be patented so the distribution of software using decoders can come at an expense. AV1 is a compression format developed by the Alliance for Open Media[29], designed to be an open, royalty-free competitor to H.265 but is still a very new format so support in software at this current time is scarce.

Audio files do not need to store as much data and as such popular support between software has remained the same for longer periods of time compared to video. Support for the MP3, AAC and PCM (commonly contained as a WAV) compression formats were chosen for this project, for the same reasons as H.264 was chosen for video. The specification originally excluded AAC but MP4 videos commonly contain audio tracks in AAC format as they are both part of the MPEG-4 standard.

5 Implementation

5.1 Data Management

The Vuex state management library was used to manage both the state of user interface elements and the data of the currently open project. The store was accordingly split up into two modules: the UI module and the project module. The UI module is responsible for keeping track of which modal dialog boxes are open and to store information about object currently being dragged by the user's cursor. The project module handles the storage of media files, the timeline and project metadata.

Some method for persisting data across sessions was required for users to be able to load existing projects. Vuex stores the state as a single JavaScript object that can easily be converted to a JSON string and back. This allows the entire state to easily be serialized to a file. File reading and downloading systems were implemented so the user can save their project at any time.

One of the main difficulties encountered was that large media files are preferably not stored redundantly on the user's system. Native video editors solve this problem by creating links to media on the file system rather than copying files to a new location. Web browsers scripts do not have arbitrary access to the file system but it is possible for the user to select files to be accessed per session.

Browsers include support for an almost convoluted variety of APIs for saving data. From the non-standard file system API, to the deprecated WebSQL database, to the newer asynchronous NoSQL-based IndexedDB approach, there is a diverse set of methods. The problem with all of these solutions is that this data is saved separately on the disk, it is not possible to maintain access to a file on the user's system across sessions. For this reason, media files are not included with project data and a system was implemented so

that users must re-import media when the project is loaded from file.

State management facilitated the development of an undo feature by saving snapshots of the project's state for each executed action. Performing undo and redo functions via the toolbar or keyboard shortcuts replays the snapshots by replacing the entire state. A hard limit is set on the number of steps that can be saved so the program doesn't run out of memory.

5.2 Video Processing

5.2.1 FFmpeg Compilation

The ffmpeg.js library is built using a Makefile which contains the commands and flags used to compile the various dependencies. It outputs multiple versions of the library with varying levels of support and optional web worker integration. Experimenting with the library revealed several drawbacks of ffmpeg.js. It compiles to asm.js, a non-standard specification inferior to WebAssembly. The pre-compiled library does not support the FFmpeg filters needed to implement some features, namely filters needed to set playback timestamps and generate blank video and audio. Another defect found in ffmpeg.js is that it no longer compiles with recent versions of Emscripten, investigation revealed that this was due to a combination of changes within Emscripten and ffmpeg.js's lack of conformance with Emscripten's guidelines. Emscripten's documentation states that the pre-js command line flag should not be used to enclose the compiled code in an inner function scope[30].

It was decided that a custom version of ffmpeg.js would be compiled so that the performance could be maximised by utilising the latest versions of FFmpeg, its dependencies and Emscripten. In addition, this approach would have better reliability with conformance to Emscripten's guidelines and also the experience gained could be used to compile other C projects. The resulting solution was heavily influenced by ffmpeg.js but extra care was taken to identify the necessity of each compilation flag.

The newer version of FFmpeg had differences in its compilation configuration and required modifications to these flags. The same process was carried out on the libx264 library[31], an FFmpeg dependency to add support for H.264 video encoding. The only

other dependency that was required to fulfil the specification was zlib—FFMpeg uses this library to implement DEFLATE compression for PNG support. Fortunately, Emscripten inherently supports zlib as a commonly used library so compiling it separately wasn't necessary.

The resulting solution had many benefits over ffmpeg.js.

- The library was compiled as a Wasm module rather than asm.js.
- Emscripten's `MODULARIZE` flag was used so that FFmpeg could be instantiated multiple times.
- Only the necessary functionality and dependencies were included, resulting in a slimmer output size.

A web worker was used to allocate the Wasm module to its own thread so that the application remained responsive during the exporting phase. Support for multi-threading was also tested with the custom build of FFmpeg as it would have been a breakthrough for performance. FFmpeg and libx264 both have support for pthreads, the only threading model supported by Emscripten, making use of SharedArrayBuffers. Disappointingly, the module encountered errors during execution with threading enable. Emscripten's support of pthreads has specific deficiencies[32] and it was concluded that this was the cause of these errors. The exhaustive development cost to investigate and resolve this problem was outweighed by implementing features from the specification.

5.2.2 Exporting with FFmpeg

An algorithm was designed to translate the representation of the timeline from JSON format to a string of FFmpeg command parameters. A more elegant approach would be to interface with FFmpeg's modules (libavcodec, libavfilter, etc) individually but using these would require developing a custom API to interface between C and JavaScript, a project in and of itself. Additional licensing considerations would also have to be to accounted for as the software would then be considered a derivative work and would be subject to GPL licensing terms. While generating an FFmpeg command is not the most elegant approach to the problem, the alternatives are significantly more complex.

FFmpeg makes use of two concepts known as filter graphs and filter chains for representing video operations. Filter graphs are made up of a sequence of filter chains which are

a sequence of filters. The inputs to the command were computed as dependencies from media in the timeline. For video, filter graphs are built up from chains of overlay filters paramertized at a specific time and position, starting from a blank, black video. For audio, filter graphs are built up from chains of audio mixing and delay filters, starting from silence. Audio is taken from both audio files and audio tracks in video files. When the FFmpeg command is run, the periodic progress output is used to display a progress bar to the user; this was only possible because FFmpeg runs in its own worker thread.

5.2.3 Previewing with HTML video and canvas

Cursory testing of Wasm compiled FFmpeg showed that it was much slower than the natively compiled version. Performance testing was then carried out on different configurations of FFmpeg to identify the causes of these slowdowns with results presented below. The test involved re-encoding a H.264 video two seconds in length with a resolution of 2560×1440 without any filters applied. It was mostly a measure of performance of the libx264 encoding library.

FFmpeg Version	Encoding Time
WebAssembly	11.5s
Native - Configuration A	5.5s
Native - Configuration B	1.8s
Native - Default	0.6s

Configuration A - FFmpeg compiled with the same compilation flags as the WebAssembly version.

Configuration B - FFmpeg compiled with the same compilation flags as the WebAssembly version but with multi-threading enabled.

The considerable performance difference of the configured versions and the default version indicate that the configuration flags used to compile the WebAssembly version had a significantly negative effect on encoding times. By comparing configuration B to the default version, it is implied that the hardware specific optimizations which needed to be disabled incurred a 3x performance cost. The difference between configuration A and

configuration B emphasises another 3x performance loss for disabling multi-threading—this is on a quad-core i7 6700k processor. Therefore, the actual performance difference between C and Wasm was around 2x. A consideration of this result is that Wasm must be compiled by the browser before it is executed; by caching a compiled version it is possible to improve the start up time but this technique was not explored in this project. The overall slowdown compared to the default native version was around 19 times—it was recognised that encoding the full video using FFmpeg for previewing purposes would be unsatisfactory and a faster solution was required.

One conceived solution was to only render the segment of the video that the user is currently previewing by exporting the video in short, lower resolution snippets. Premiere Pro uses a similar technique by only rendering video which is played while caching it for smoother, subsequent playback. If it were possible to use FFmpeg's libraries programatically, this may have been the best solution because it could have been used to render frames individually. The current implementation suffers from the performance overheads of initiating a command line interface and in combination with the already considered performance deficits, it would have been difficult to guarantee responsive playback. Another solution was to make use of the possibilities discussed in the Research, Video Processing, Alternatives section by including another library but were overlooked in favour of the performance brought about from using the browser's video decoder.

The employed solution makes use of the HTML video and canvas elements to draw video frames to the canvas. The HTML video element is capable of displaying video from a file source and can be controlled using JavaScript. The drawImage canvas function can take a video element as input to draw the frame displayed by the video to the canvas. The trade-offs for this particular solution were adequate and guaranteed the largest performance boost.

Advantages of the approach.

- This solution makes use of the video decoder included in the browser which is potentially faster than any solution written in JavaScript or WebAssembly because it can make full use of the hardware available.
- Using the canvas element means any of the canvas's drawing functions can be used to include graphics into the video or add filters. WebGL can also be used for wider flexibility and better performance with shaders.

- CCapture.js can be used to render video, supporting WebM but not H.264 compression, making it possible to completely replace FFmpeg.

Disadvantages of the approach.

- The video element cannot be read from a web worker as the DOM is only accessible on the main thread.
- There is an unknown overhead of using perpetually instantiated HTML video elements.
- This solution does not involve audio and would have to be implemented separately.
- Only video formats supported by the browser can be used.

A few of the disadvantages could be addressed using additional features. FFmpeg could be used to convert incompatible videos to a format supported by the browser when they are imported. Audio support could be incorporated separately using WebAudio APIs. One of the difficulties that couldn't be overcome was the additional work instigated from using multiple video processing systems. Since FFmpeg was still used for exporting, it is separate from the canvas based system. Video features such as transformations, text and transitions need to be implemented twice, for each system respectively. Despite the increased workload, the return in performance and responsiveness was considered as a beneficial trade-off.

5.3 User Interface

Figure 5.1 shows the final dependencies between all of the user interface components. Vue components and Vuex state management provided useful mechanisms for communication in almost all situations. There were two scenarios where this architecture wasn't adequate: context menus and keyboard shortcuts. These systems were implemented using a separate event driven model because the storage of global information was unnecessary.

5.3.1 Media Browser

The importing process for media was similar to the importing process of projects. The main difference was identification of the type of media accomplished by checking the

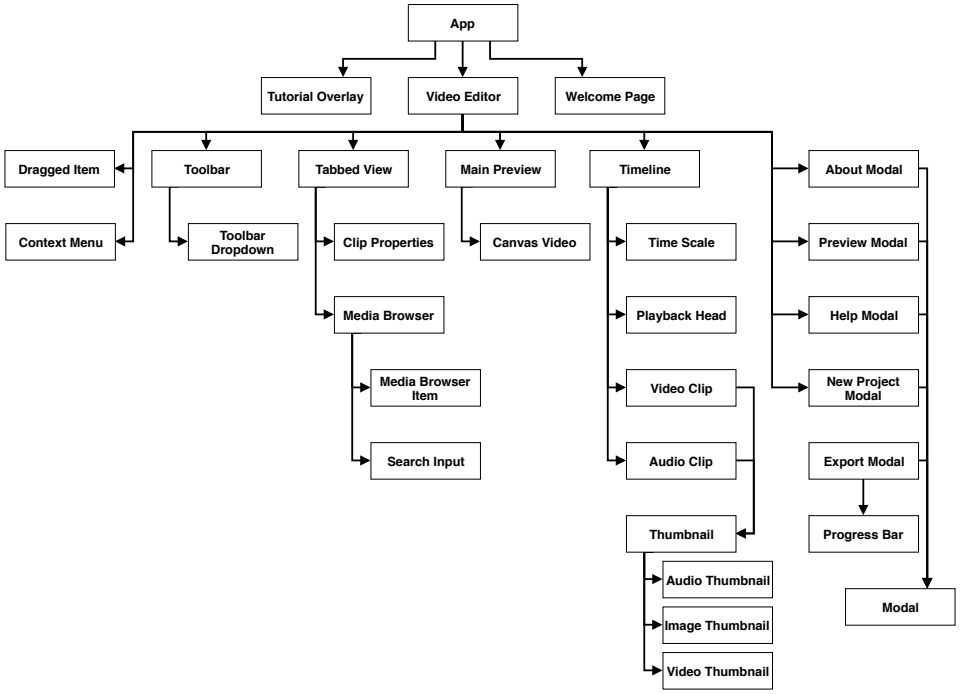

Figure 5.1: Dependency graph for UI components

MIME type of the file. The program categorises the media by file type as video, image or audio content. Search and category filters were built into the media browser so that users could more easily find imported media. Thumbnails for images were simply implemented as image elements while video thumbnails are drawn from the first frame of their video; the same system is reused for the timeline.

5.3.2 Timeline

The timeline is a hub of interconnected parts making it the most intricate user interface component to implement. Scaling functionality was added so that the timeline can be used as an overview or for precise editing, this required taking into the account the frames per second of the video as the length of clips are measured in the number of frames they are displayed for. A ruler displays time divisions and frames so the user can decipher the particular part of the video being represented. This was implemented using a canvas element so that markings could be drawn programatically according to the scrolled amount, timeline scale and frames per second of the video.

When the user adds, moves or modifies the length of clips in the timeline, it is possible

for multiple clips to overlap on the same layer. To account for this, each modification invokes a conflict resolution process to trim or overwrite clips which are in the way of a new clip—Figure 5.2 shows the situations where this is necessary. Selection functionality was also added so that users may interact with multiple clips simultaneously by clicking on clips while holding the shift key. This allows multiple clips to be moved around, deleted or copy and pasted which significantly speeds up productivity.

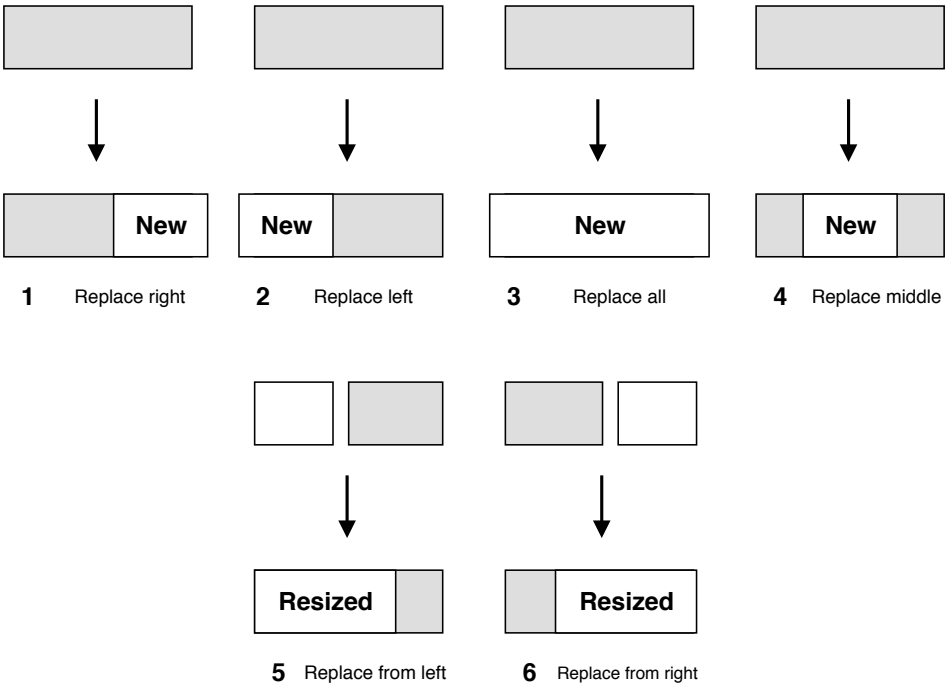

Figure 5.2: Six scenarios that must be resolved when clips are added or resized

5.4 Documentation

This project makes use of the JSDoc[33] documentation format. Similarly to testing, documentation was not prioritised due to the exploratory nature of the project so documentation was mostly foused on state management functions and more involved systems such as the FFmpeg translator. One of the advantages of using JSDoc with VSCode was its interoperability with VSCode's TypeScript server. VSCode uses type annotations from JSDoc to infer the types of some of the variables to aid development with with better code completion and optional type errors. This was used in turn with a TypeScript definition file to externally define some interfaces, while not being a full

replacement for TypeScript, provided additional flexibility for documentation.

5.5 Networked Collaboration

A rejected extension idea excluded from the specification was to leverage the web aspect of the application to implement a collaborative feature which allows multiple users to work on the same document from separate devices in real time. This idea originated from other products such as Google Docs[34] for writing documents and Visual Studio Live Share[35] for code reviews and pair programming. The great benefit of a collaboration feature is that it enables instant feedback to be given while editing to streamline reviewing of work. The idea was dismissed since there was concern that such a feature would be too complicated to architect in combination with an already complex application in the given amount of time. However, near the end of the development the idea was revisited and a functional prototype was tested.

To facilitate the communication between multiple clients, the WebSockets API was used. This API provides a means of bidirectional communication between a client and server through the browser. It was used to implement the prototype because of its ease of setup but an external server is required. The server has the basic role of echoing received messages to all other clients. An alternative approach would be to use WebRTC peer-to-peer data channels, involving a slightly more in-depth implementation but would allow clients to communicate without a server.

The Vuex state management pattern is designed to be easily traceable and provides listeners for mutations and actions. Since all modifications to the state are enacted through mutations, it allows exactly the same state to be replicated by executing mutations in the same order. A listener is used to broadcast mutations in JSON format to all of the other connected clients which are then applied to the state. The separation of UI and project state made it simple to only synchronise the project data; the user interface can be controlled individually by each user. This architecture was the reason it was possible to implement such a future late into development with little difficulty.

There are a few limitations of this feature in its current state. Files are not transferred over the network so they must be shared separately by users and must be linked in the same way that is necessary after loading a project. Another problem is that there is

no resolution of conflicts so when mutations are executed simultaneously on different clients, the state could become unsynchronised.

6 Testing

6.1 Unit Testing

Unit tests were carried out using the Jest[36] testing library. There are a plethora of JavaScript testing libraries available, this particular choice was justified by its capability of testing Vue.js components. Jest's ability to identify lines of the program or branches which were not covered by the tests helped achieve better code coverage. Since the project involved exploring different techniques, it was difficult to write unit tests to entirely cover a continually changing codebase. Thus, the unit tests were mostly written for the state management mutations and actions which were unlikely to change. As mutations and actions are already discrete, clear and purposeful functions, they were suited for unit tests. The conflict resolution process for modifying the timeline was implemented in the state management so unit tests to cover different scenarios greatly helped with its reliability. The well-tested state eased debugging as bugs were much more likely to be caused by user interface code.

6.2 Stress Testing

Stress tests were performed on the software to ensure that the application could still perform well under reasonably sized work loads. This involved importing large media files of different types and observing how it was handled by the previewing and exporting systems. One problem that was discovered with large video files was that during the exportation phase, the program would consistently crash if the sizes of the input files were too large. It was hypothesised that this was caused by the 4GiB memory limit of WebAssembly[37], a restriction that cannot be overcome until 64-bit support is added to Wasm.

The solution was fortunately rather simple; Emscripten uses the MEMFS file system by default where files are copied into the memory to be used by the Wasm module, fast but not suited for this situation. Emscripten provides a selection of file systems, the most suitable for this use case was WORKERFS, providing web workers with access to file objects. Transitioning to WORKERFS resolved these issues as files were read from the disk rather than from memory. Memory limitations can be difficult to work around as JavaScript doesn't have the clearance to know the amount of memory accessible for security reasons.

7 Evaluation

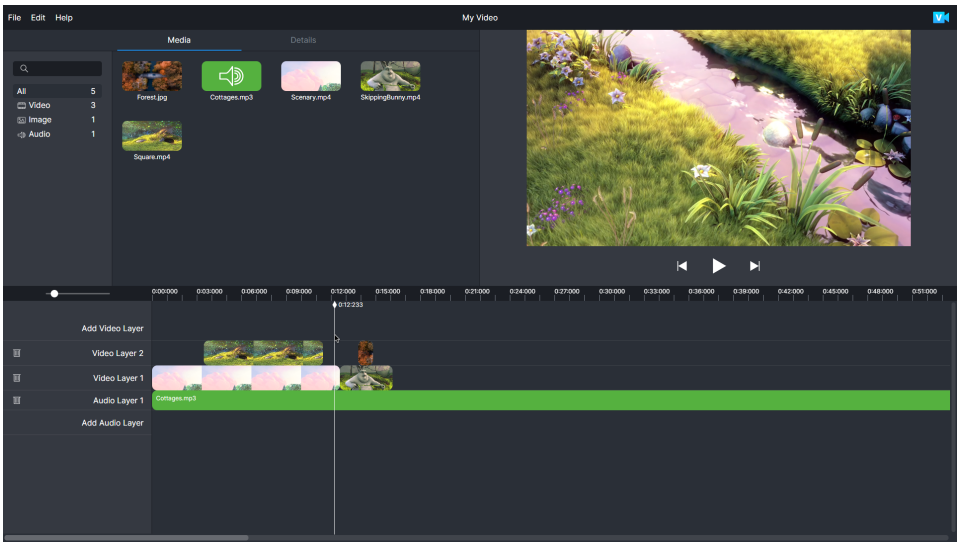

Figure 7.1: Screenshot of the main timeline view

Most of the specification was successfully realised at a fulfilling standard of quality. The software achieves the goal of allowing users to import media, edit their content and export it to a video file entirely in the browser. The application stays true to its target audience by providing a familiar interface for video editing while still incorporating a tutorial and help centre to assist new users. The only parts of the specification which were not achieved were objectives 7 and 9—support for text and transitions. It was difficult to estimate the amount time to allocate each feature in the planning phase because of unknown intricacies that could arise during implementation and testing; further delays were caused by the dual video processing system. The decision to focus the testing and documentation on state management heavily bolstered the software's reliability by providing a solid foundation to the software while still accommodating the investigation of various web technologies. In terms of performance, the software is very usable for practical applications but there are still many optimisations to be made, major and

minor.

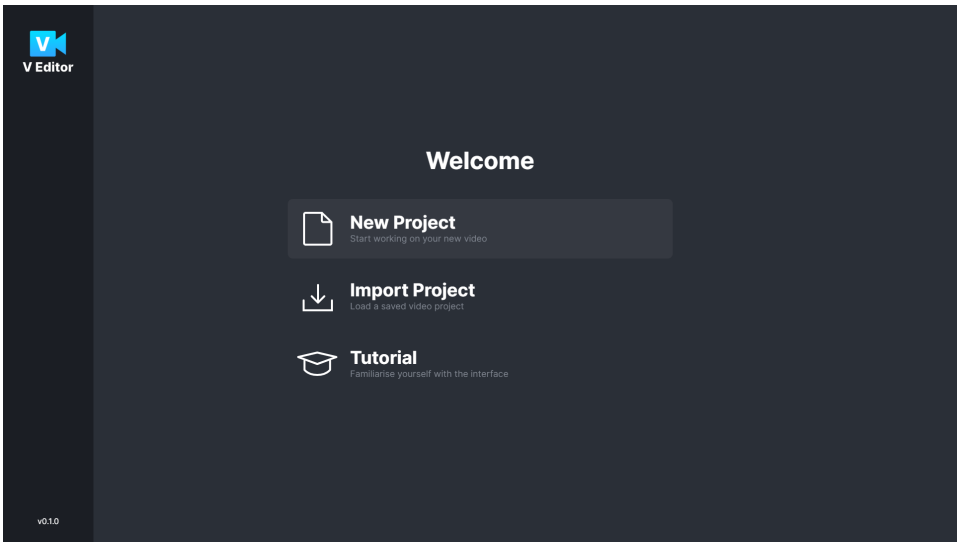

Figure 7.2: Screenshot of the welcome page

During development, UI and UX design became a major focus of the project. These were two key elements for creating high quality software and lead to the discovery of more practical and problem specific issues. An alternative focus on the analysis of web technologies and performance testing would have accompanied a better comprehensive study on web applications but would have negatively impacted the usability of the software. The project required balancing software development and research to provide a unique outlook on the subject.

For the most part, V Editor's current feature set does not expand further than the other software studied: Premiere Pro, iMovie or ClipChamp. To reach a similar level of features to iMovie and ClipChamp, video filters, text and transitions features would need to be added. There are minor exceptions to this: V Editor has no layer restriction like iMovie and is not limited by a minimum length for clips like ClipChamp. The most outstanding feature that V Editor offers is the prototyped real time collaboration system; other examples of this feature for video editing were unidentified. With the further extension of a JavaScript plugin system, this software would be characterised by having truly unique features among other video editing competitors.

7.1 Project Management

Git was used for version control from the start of the project to the end with excellent success. It was relied upon frequently for testing new features with an easy way of rolling back to a previous version. It was also used as a tool for synchronising the code between multiple computers which helped with testing the software on multiple systems. Importantly, Git meant the code was backed up in multiple locations and on GitHub[38] for reliable protection against data loss.

One aspect of project management that could have been improved was planning. The planning of the specification was sufficient but as more in-depth features became integrated in the software, the project would have benefited from a formal method of prioritising features and tracking which were still in-progress and which were yet to be implemented—this could have been done using a Kanban board. This situation improved near the end of the project when GitHub's issues feature was used to keep track of bugs.

8 Further Work

8.1 Feature Additions

8.1.1 Text and Transitions

These objectives in the specification were not attained due to time constraints; both warrant separate implementations for FFmpeg and for canvas. One of the difficulties of rendering text was the necessity of another FFmpeg dependency. Furthermore, FFmpeg would need access to the font files which are not available from WebAssembly by default. It would be trivial to add text to the canvas based rendering system using the canvas's text drawing function but this feature alone would be purposeless.

8.1.2 JavaScript Plugins

A JavaScript plugin system was originally envisioned in the specification which would give users the ability to write scripts to programmatically alter the video output. Web extensions are a popular aspect of web browsers as it allows users to control their browsing experience. This feature would play to the strengths of JavaScript as a scripting language and would allow for the creation of exceedingly flexible effects for users who desired precise control and freedom.

8.1.3 Video Filters

Video filters are a common feature of video editors which enact some operation on each video frame to apply a specific effect. Examples of video filters include grayscale, brightness, saturation and blur. Filters are often easily parallelisable and benefit from

being implemented as a shader which could present an issue for FFmpeg as Wasm only has control over the CPU.

8.2 Performance Improvements

There are a number of certain of areas of the software which could undergo performance optimisations to further improve the user experience. The biggest foreseeable performance gain is multi-threading the video exporting process. Compiling FFmpeg with Emscripten's threading support was unsuccessful and while it is possible that this will be resolved by Emscripten in the future, there are other methods which could be used to achieve this goal. The program could potentially separate the video into equally long sections, then execute FFmpeg in separate web workers for each of these sections, concatenating the result at the end. There is naturally some overhead to consider but if the majority of the work is carried out in multiple threads, it would be a great improvement on export times, especially for high core count CPUs.

Native FFmpeg is capable of utilising the GPU to accelerate certain operations including video encoding. This is possible through AMD's VCE[39] or Nvidia's NVENC[40] technologies, neither of which are accessible through the browser. Hardware accelerated video encoding on the web is unforeseeable at this current time as VCE and NVENC are too hardware specific.

Another general improvement that would increase the responsiveness of the program would be to cache more of the computed data. Preserving previously drawn video frames would allow for less intensive playback. Caching thumbnails of imported media so they are not reprocessed for each video clip in the timeline would eliminate unwarranted overhead. While browsers are capable of caching resources retrieved from the network, developing such a system in JavaScript could be difficult as it is not possible to access system information due to security reasons.

8.3 Usability Improvements

8.3.1 Audio Preview Support

The drawback of the decision to use a canvas based approach for previewing video meant that audio had to be implemented separately. This could have been possible using the WebAudio API but time restrictions meant that this feature was not completed. Audio is understandably an important aspect of video editing and the absence of any audio queues makes it difficult to synchronise audio with video content.

8.3.2 Data Persistence

One of the greatest inconveniences with the application is the lack of persistence between sessions. Users must save their work to a file but it does not contain media so once the project is imported, all of the media files must be imported again. This is an unconventional restriction without a foreseeably adequate solution in the browser. Therefore, the suggested future of this software is to employ a hybrid technique discussed in the Design, Application Strategy section and add an additional deployment profile which uses a local server or Electron for file reading.

8.3.3 Transformation Interface

The position and size of video clips can be set by the user using the clip details panel by inputting numerical values. This method is useful for precise control but is unintuitive to use without a graphic interface. A more graceful approach would be to allow users to control the video clip inside the video preview by moving it using the cursor.

9 Conclusions

This project provided deep, practical insights into the competencies and constraints of web applications. The open nature of the internet has encouraged collaborative development of web specifications and an abundance of open source libraries and frameworks. This rationale has set a low barrier of entry for utilising these technologies as resources are exceedingly accessible. The existence of Node.js and Electron open up possibilities outside of the browser, appointing JavaScript as a flexible and practical language to work with. The prevalence of JavaScript for web development has lead to the existence of a thriving ecosystem of tools, providing developers with a wide variety of choice. Tools like Vue.js facilitate rapid development of user interfaces. Webpack's hot module reloading is a productivity luxury for testing.

A criticism of web development is the rapid pace at which new libraries and tools are popularised, complicating the learning process for web developers. A similar situation is the volatile nature of web APIs as they are occasionally deprecated in favour of a better alternative, requiring applications to undergo re-architecture. The numerous methods of data storage, WebAssembly replacing asm.js and obsolete JavaScript features are examples of this. There are certain problems with the JavaScript language that make it unsuitable for writing scalable code with few guarantees of type safety. Solutions like TypeScript exist to patch these problems but still indicate issues in the language.

Front-end development can be heavily restricted by the APIs available in the browser—a necessary precaution against unsafe websites. This can sometimes to lead to overly complicated implementations for seemingly simple functionality. Software which relies on specific hardware, low-level network access or interprocess communication are completely infeasible on the web for this reason. While specifications for new web APIs are in constant development, there are many impractical use cases currently present.

This project concludes that while web applications promise a remarkable number of

merits, they are not the be all and end all of software development. There are still many circumstances where this approach is inappropriate, although this is not necessarily always going to be the case. Out of all of the technologies explored, WebAssembly has the most fascinating potential for influencing the future of software development. The power that it provides and its use cases inside and outside the web impose optimistic promises. The tools available today are competent and it is likely that many more are still to come. The future of technology is unpredictable, but the direction of the web will assuredly be interesting.

10 Acknowledgements

I wish to thank my supervisor for the helpful project advice and direction.

I would like to thank my family and friends for their support during the year.

10.1 Developer Resources

- Can I Use, https://caniuse.com/ - A resource for checking feature support between browsers.
- Figma, https://www.figma.com - The tool used for UI prototyping and design.
- MDN, https://developer.mozilla.org/en-US/ - Comprehensive documentation and examples of web technologies with up-to-date information on experimental features.
- VS Code, https://code.visualstudio.com - An excellent code editor for web development.
- Big Buck Bunny, https://www.bigbuckbunny.org/ - Video content used for testing and demonstration.

11 Glossary

CSS Cascading Style Sheets is a language for describing the presentation of a HTML document.

DOM The Document Object Model is a programming interface for interacting with HTML.

Front-End Application A client-side application created using HTML,CSS and JavaScript.

HTML Hypertext Markup Language is a language used for semantically defining the contents of a web page.

JavaScript A programming language most used for scripting web pages.

Node.js A runtime for executing JavaScript outside of the browser.

Web Application A piece of software created using web technologies.

Web Technologies Systems, languages, protocols or frameworks designed for developing software related to the web browser.

12 Legal Considerations

Libraries and frameworks used have their licenses adhered to. Vue, Vuex and clone-deep libraries are all released under the MIT license. FFmpeg includes libx264 as a dependency so the compiled version is subject to the GPL-2.0 license. V Editor utilises FFmpeg as an executable and is therefore not considered a derivative work so is not required to be sublicensed as GPL.

Bibliography

[1] A. Haas, A. Rossberg, D.L. Schuff, B.L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, J. Bastien, Bringing the web up to speed with WebAssembly, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2017, ACM Press, Barcelona, Spain, 2017: pp. 185–200. doi:10.1145/3062341.3062363[1].

[2] iMovie, Apple, 2019. https://www.apple.com/uk/imovie/ (accessed April 21, 2019).

[3] Premiere pro, Adobe, 2019. https://www.adobe.com/uk/products/premiere.html (accessed April 21, 2019).

[4] Clipchamp create, Clipchamp, 2019. https://clipchamp.com (accessed April 21, 2019).

[5] P. Kocher, J. Horn, A. Fogh, {.D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, Y. Yarom, Spectre attacks: Exploiting speculative execution, in: 40th IEEE Symposium on Security and Privacy (S&P'19), 2019.

[6] The Khronos Group, WebGL - OpenGL ES for the web. The khronos group, (2011). https://www.khronos.org/webgl/ (accessed April 27, 2019).

[7] WebAssembly Contributors, WebAssembly, (2019). https://webassembly.org/ (accessed April 25, 2019).

[8] Unity Technologies, WebAssembly is here! – unity blog. Unity technologies blog, (2018). https://blogs.unity3d.com/2018/08/15/webassembly-is-here/ (accessed April 21, 2019).

[1] https://doi.org/10.1145/3062341.3062363

[9] Epic Games, Unreal engine 4 documentation - developing HTML5 projects, (n.d.). https://docs.unrealengine.com/en-us/Platforms/HTML5/GettingStarted (accessed April 21, 2019).

[10] A. Crichton, Multithreading rust and wasm. Rust and WebAssembly, (2018). https://rustwasm.github.io/2018/10/24/multithreading-rust-and-wasm.html (accessed April 27, 2019).

[11] WebAssembly Contributors, WebAssembly/gc, (2019). https://github.com/WebAssembly/gc (accessed April 28, 2019).

[12] Emcsripten Contributors, Emscripten 1.38.27 documentation, (2019). https://emscripten.org/ (accessed March 10, 2019).

[13] A. Zakai, Emscripten: An LLVM-to-JavaScript compiler, in: Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications Companion, ACM, New York, NY, USA, 2011: pp. 301–312. doi:10.1145/2048147.2048224[2].

[14] Angular, Google, 2019. https://angular.io/ (accessed April 21, 2019).

[15] React – a JavaScript library for building user interfaces, Facebook, 2019. https://reactjs.org/index.html (accessed April 21, 2019).

[16] E. You, Vue.js, 2019. https://vuejs.org/ (accessed March 12, 2019).

[17] E. You, Vuex, 2019. https://vuex.vuejs.org/ (accessed March 12, 2019).

[18] Flux application architecture for building user interfaces, (2015). http://facebook.github.io/flux/index.html (accessed November 25, 2018).

[19] TypeScript, Microsoft, 2018. http://www.typescriptlang.org/ (accessed November 21, 2018).

[20] Webpack, (2019). https://webpack.js.org/ (accessed April 27, 2019).

[21] K. Hiiragi, Ffmpeg.js, 2018. https://github.com/Kagami/ffmpeg.js (accessed March 10, 2019).

[2] https://doi.org/10.1145/2048147.2048224

[22] J. Sanchez, CCapture.Js, 2019. https://github.com/spite/ccapture.js (accessed April 21, 2019).

[23] M. Bebenita, Broadway.js, 2019. https://github.com/mbebenita/Broadway (accessed April 21, 2019).

[24] M. Greenblatt, Chromium embedded framework, 2019. https://bitbucket.org/chromiumembedded/cef (accessed April 21, 2019).

[25] Figma, Figma, 2019. https://www.figma.com/ (accessed April 21, 2019).

[26] Google, Progressive web apps web. Google developers, (2019). https://developers.google.com/web/progressive-web-apps/ (accessed April 26, 2019).

[27] Google, New in chrome 73 web. Google developers, (2019). https://developers.google.com/web/updates/2019/03/nic73 (accessed April 26, 2019).

[28] Feature request - HEVC support - chromium - monorail, (2019). https://bugs.chromium.org/p/chromium/issues/detail?id=684382 (accessed April 21, 2019).

[29] Alliance for open media, (2019). https://aomedia.org/ (accessed April 21, 2019).

[30] Emscripten Contributors, Emscripten compiler frontend (emcc), pre-js — emscripten documentation, (2019). https://emscripten.org/docs/tools_reference/emcc.html#emcc-pre-js (accessed April 25, 2019).

[31] VideoLAN, X264, the best h.264/AVC encoder. VideoLANOrganisation, (2019). https://www.videolan.org/developers/x264.html (accessed April 26, 2019).

[32] Emcsripten Contributors, Pthreads support — emscripten documentation, (2019). https://emscripten.org/docs/porting/pthreads.html#special-considerations (accessed April 26, 2019).

[33] JSDoc, (2019). http://usejsdoc.org/ (accessed April 21, 2019).

[34] Google docs, Google, 2019. https://www.google.co.uk/docs/about/ (accessed April 28, 2019).

[35] Visual studio live share, Microsoft, 2019. https://visualstudio.microsoft.com/services/live-share/ (accessed April 28, 2019).

[36] Jest, Facebook, 2018. https://jestjs.io/index.html (accessed November 26, 2018).

[37] Features to add after the MVP - WebAssembly, (2016). https://webassembly.org/docs/future-features/#linear-memory-bigger-than-4-gib (accessed April 21, 2019).

[38] GitHub. GitHub, (2019). https://github.com (accessed April 21, 2019).

[39] AMD, Introducing the video coding engine (VCE) - AMD, (2016). https://web.archive.org/web/20160604071338/http://developer.amd.com/community/blog/2014/02/19/introducing-video-coding-engine-vce/ (accessed April 21, 2019).

[40] Nvidia, NVIDIA VIDEO CODEC SDK. NVIDIA developer, (2013). https://developer.nvidia.com/nvidia-video-codec-sdk (accessed April 21, 2019).